

Dwarpal user manual

This manual is for a developer, or an AI coding agent, about to integrate `@briqvent/dwarpal` into a web application. Read it fully before planning the integration. The contract and its guarantees are in the [design](#); the SQL design and failure analysis are in the [RBAC low-level design](#).

Status: this manual describes contract 0.5. The package is not released yet; commands and entry points are those the release will provide.

1. What Dwarpal is

A reusable authentication and authorization layer on **Supabase Auth (Free plan or above)** for websites with a static or server-rendered front end and, optionally, a Node.js back end. It provides:

- Email and password sign-up with verification, sign-in, sign-out, forgotten and reset password, Google sign-in, and TOTP multi-factor authentication.
- Ready-made, brandable screens for those flows, or a headless controller if you build your own pages.
- A server library that turns a request's bearer token into a verified **Principal**: who the user is, which roles they hold for your site, and which permissions those roles give them.
- A database layer in your Supabase project that stores clients, roles, permissions, enrollments and memberships, written only through audited SQL functions.
- Helper functions you may call from your own Row Level Security policies.
- A command-line tool for the operator, `auth-kit`: migrations, client registration, permission model, first manager, diagnostics.

2. What Dwarpal is not

- **Not a service.** There is no server to run and no network API of its own. The server part is a library you import into your Node process. All calls go from your browser or your server to your Supabase project over TLS.
- **Not a row-level authorization system.** It answers "may a buyer read orders". It never answers "may this buyer read order 123". You keep that check: in code after the permission check, or in your own Row Level Security policies if your data lives in Supabase Postgres.
- **Not a live-revocation guarantee for direct database access.** Only requests that pass through your Node server and `resolveSession` get the live session check. Section 8 explains the two paths.
- **Not an admin console.** Roles and permissions are defined in a file applied by the operator. Managers grant and revoke memberships through your own pages using the library; there is no built-in management UI.
- **Not a store of your business data.** It never holds customer, lead or order references. If a user must be linked to a historical record, your application stores and audits that link.
- **Not an adapter for any particular application.** There are no application-specific helpers or hooks. You build the actor your code needs from the `Principal` the server returns. The kit does not know what a lead, an order or a customer is.
- **Not a cookie-session system.** Tokens live in the browser's local storage via the official Supabase client. If you need server-managed cookie sessions, this kit does not provide them.

3. Concepts

Term	Meaning
Client	One website or application registered in the project by a <code>client_id</code> . Default deployment: one Supabase project per website.
Identity	A Supabase Auth user. The stable key is <code>userId</code> . <code>verifiedEmail</code> is the email Supabase has confirmed; it is not the same thing as any contact email the user typed into a form.
Session	Issued and stored by Supabase Auth. Access tokens last a configured time (30 minutes recommended); refresh tokens rotate. The server library keeps no session state.
Role	A named row your operator defines for your client, such as <code>supervisor</code> , <code>agent</code> , <code>buyer</code> , or <code>owner</code> , <code>editor</code> , <code>member</code> . Names are yours.
Reserved flags	Three flags on every role that the kit itself enforces: <code>self_assignable</code> (granted automatically at a user's first enrollment), <code>manages_members</code> (holders may grant and revoke non-manager roles), <code>mfa_required</code> (the role only counts in an MFA-verified session). A role can never be both self-assignable and a manager.
Permission	A named key your operator defines, recommended shape <code>resource:action:scope</code> , such as <code>orders:read:own</code> and <code>orders:read:any</code> . Roles map to sets of permissions.
Membership	A user holding a role for a client. A user may hold several roles for one client.
Enrollment	The one-time event, per user and client, in which a verified user automatically receives every <code>self_assignable</code> role. It is recorded durably and never repeats, so a revoked role stays revoked until a manager grants it again.
Active role	A held role whose <code>mfa_required</code> flag is satisfied by the current session. Every authorization decision, in the library and in the SQL helpers, uses active roles only.
Principal	The verified result of resolving a request on the server: identity, session, memberships and the access block (enrollment, roles, active roles, permissions, MFA status) for your configured client only.
Operator	The trusted administrator of your deployment: whoever holds your project's secret key and runs the CLI. Owns the permission model and manager promotion.
Manager	A user holding a <code>manages_members</code> role. Manages people, not the model. Manager actions need at least one active manager role: if the only manager role you hold requires MFA, you must be at MFA level to grant or revoke; if you also hold a manager role without that flag, that role suffices.

4. Rules of engagement

These are the rules the kit relies on. Breaking them is not supported.

1. **The browser gets only the project URL and the publishable key.** Never ship the secret key, the Management API token or provider secrets to a browser, a log line or a repository.
2. **Your web server never holds the secret key.** It resolves sessions with the publishable key and the request's own token. Only the operator's shell holds the secret key, in a local environment file outside any repository.
3. **Sensitive records are read and written only through your Node server**, which calls `resolveSession` (live session check) and then `requirePermission`. Anything the browser reports about the signed-in user is display state. Direct browser access to Supabase tables is allowed only for public or low-risk data, and only with the limits in section 8.
4. **Your server's `clientId` comes from your configuration, never from the request.**
5. **One permission key means one thing.** Use distinct keys for a user's own rows and for all rows (`orders:read:own`, `orders:read:any`). `own` always refers to the row being acted on, never to the target of the action. An action whose narrow scope is "the principal as the target", such as assigning a lead to oneself, uses `:self` (`leads:assign:self`), and your guard compares the target user id with the principal's. Check the broad key first with `can`; if it is absent, require the own key and then check

ownership yourself. Never derive "is staff" from the list of held roles; use `activeRoles`, `can` or `requireRole`, which only look at active roles.

6. **Never look up historical records by email.** A verified email is proof of who the user is now, not proof that they own an old lead or order. Store an explicit, audited link row in your own database when a manager links a user to a record, and provide an unlink path.
7. **Store only `userId` as the reference to a user.** Do not store emails, tokens or session ids as keys.
8. **Do not write to the kit's tables directly.** Use the functions through the library or CLI. The secret key can technically bypass this; doing so breaks the audit trail and the diagnostics will report it.
9. **Define the model as a file in your repository and apply it with the CLI.** Business owners and managers do not edit the model. Review the dry-run diff before applying. If you ever change the model from the SQL editor, run `export-model` and commit the result before the next apply.
10. **Enrollment is once.** To remove a user's access, a manager revokes the membership; it stays revoked across sign-ins. To restore it, a manager grants it again. Do not build flows that expect sign-in to "refresh" roles.
11. **Treat `unavailable` as a denial with a retry hint (HTTP 503).** The kit fails closed when Supabase cannot be reached. Never fall back to a cached or empty principal.
12. **Keep the auth pages out of search indexes (`noindex`)** and keep public content browsable without an account.
13. **Assert the contract version at startup.** The library exports `AUTH_CONTRACT_VERSION`; refuse to start on a mismatch.

5. Integration steps

1. **Install** a pinned version of `@briqvent/dwarpal`. Run `npx auth-kit init` to get a configuration skeleton, a model skeleton and an environment example with no values.
2. **Create a Supabase project** for this website, on the Free plan or above. Enable asymmetric JWT signing keys; create the publishable and secret keys; create a Management API personal access token for the operator; enable email confirmations; set the access-token lifetime (30 minutes recommended); use `{{ .TokenHash }}` in the email templates; enable TOTP if any role will require MFA; configure custom SMTP with SPF and DKIM on the sending domain (the built-in sender is rate limited and not for production); configure Google if you use it, with the redirect URI `https://<project-ref>.supabase.co/auth/v1/callback`; and add the **exact** redirect URLs for each environment, with no wildcards. These are manual project settings; the kit cannot set them for you.
3. **Migrate:** `npx auth-kit migrate` (uses the Management API token), or paste the numbered SQL files into the SQL editor in order. Then run `npx auth-kit doctor` to confirm schema version, exposed schemas, grants and keys. This default mode inspects the database catalog with the secret key. To prove behaviour as a real user, create a disposable test user and run `npx auth-kit doctor --probe --probe-email <that user>`; the report says which mode produced each line.
4. **Register the client:** `npx auth-kit register-client --client <id> --name "<display name>" --signup open|closed`.
5. **Write the model** (`auth-model.json`) and apply it: `npx auth-kit apply-model --dry-run`, review, then `apply-model`. The model must contain at least one `manages_members` role.
6. **Bootstrap the first manager:** `npx auth-kit bootstrap-manager --client <id> --role <manager role> --user-id <uuid>`. Find the id in the Supabase dashboard, or pass `--email` and let the CLI look it up. It reads the whole user list first and refuses if the email matches no confirmed user, more than one, or if the project has more users than it will scan (about ten thousand; `lookup_incomplete`); in that case use `--user-id`. `--invite` sends an invitation to a missing user and prints the id to rerun with. The client becomes **live** after this step, and from then on the model can never leave it without a manager.
7. **Browser:** import the ESM entry and the stylesheet (all classes prefixed `ak-`), pass your validated configuration, and mount the default screens at your configured routes or drive the headless controller

from your own pages. Extend your Content Security Policy's `connect-src` with your Supabase project origin only.

8. **Server**: construct the auth server once with your `clientId`, resolve the session per request, require permissions, then apply your own ownership rules.
9. **Verify**: `npx auth-kit doctor --origin <url>` checks routes, CSP, `noindex` and the redirect allow-list against the project.

6. The model file

```
{
  "client": "example",
  "roles": {
    "owner": { "manages_members": true, "mfa_required": true,
              "permissions": ["members:manage", "posts:publish", "posts:edit:any",
"posts:read"] },
    "editor": { "permissions": ["posts:edit:own", "posts:read"] },
    "member": { "self_assignable": true, "permissions": ["posts:read"] }
  },
  "permissions": {
    "members:manage": "grant and revoke non-manager roles",
    "posts:publish": "",
    "posts:edit:own": "edit posts the user authored",
    "posts:edit:any": "edit any post",
    "posts:read": ""
  }
}
```

Validation rejects:

- a role that is both `self_assignable` and `manages_members`;
- a permission used by a role but not declared;
- a model with no `manages_members` role;
- removing a role that users still hold;
- removing a permission still mapped by a role that is not updated in the same file;
- setting `manages_members` on a role that users already hold (promotion happens only through `bootstrap-manager`);
- once the client is live, any model that would leave it with no assigned manager.

Applying is one transaction with an audit row holding the diff and the file hash. Reapplying an unchanged file changes nothing.

The dry-run diff tells you the reach of each change: a permission change says how many current holders it affects; making a role `self_assignable` says **future joiners only**, because enrollment never repeats for existing users. If existing users should get the new role, a manager grants it.

7. Server usage

```
import { createAuthServer, requirePermission, can, explain, AuthError } from
  '@briqvent/dwarpal/server';
```

```

const auth = createAuthServer({ supabaseUrl, publishableKey, clientId: 'example' });

async function handler(request, postId) {
  let principal;
  try {
    principal = await auth.resolveSession(request); // null when there is no bearer
    token
  } catch (e) {
    if (e instanceof AuthError && e.code === 'unavailable') return new Response(null, {
status: 503 });
    return new Response(null, { status: 401 });
  }
  if (!principal) return new Response(null, { status: 401 });

  const post = await loadPost(postId); // your store
  if (!can(principal, 'posts:edit:any')) { // broad branch: needs an active
role that grants it
    try {
      requirePermission(principal, 'posts:edit:own'); // throws forbidden or mfa_required
    } catch (e) {
      return Response.json({ error: e.code, why: explain(principal, 'posts:edit:any') }, {
status: 403 });
    }
    if (post.authorId !== principal.identity.userId) return new Response(null, { status: 403
});
  }
  // your business logic
}

```

Building your own actor. Read `principal.identity.userId` for the stable subject, `principal.memberships` (already scoped to your configured client) for role keys, `principal.access.permissions` for what the session may do now and `principal.access.activeRoles` for the roles that count now. Put that in whatever object your code wants; the kit ships no such object. If your application keeps links between users and records, key them by `identity.userId`, store them in your own tables and audit them yourself.

Testing your guards. Import the kit's evaluation fixture from `@briqvent/dwarpal/testing`. It exports principals for each role, MFA state and client scope that the kit itself is tested with, so your guard tests and the kit's tests agree on what a principal looks like.

Framework glue. `@briqvent/dwarpal/server/hono` exports a middleware that sets `c.var.principal` or responds 401 or 503. Hono is an optional peer dependency; nothing else in the kit depends on it.

Manager actions from your own pages run under the manager's token: `auth.grantMembership(userId, roleKey, requestId)` and `auth.revokeMembership(userId, roleKey, requestId)`. Pass a fresh UUID as `requestId` per logical action and reuse it on retry:

- the same id with the same payload returns the first result and writes nothing;
- the same id with a different payload fails with `request_conflict` and writes nothing.

This holds even when the first call changed nothing (for example `already_member`), because the kit records every request's outcome, not only mutations. An id is bound to the manager who first used it. Managers cannot grant or revoke manager roles.

House pattern. The same request-id rule is the recommended shape for your own idempotent commands: a fresh UUID per logical action, reused on retry; store a fingerprint of the operation, client, actor and canonical payload with the first result; return the stored result on an identical retry and refuse with a conflict otherwise; and write the row even when the first run changed nothing.

8. Data in Supabase Postgres: the two access paths

If some of your tables live in Supabase Postgres, you may protect them with Row Level Security policies that call the kit's helpers: `auth_kit.has_permission(client_id, permission_key)`, `auth_kit.has_role(client_id, role_key)` (active roles only) and `auth_kit.has_aal2()`. They return `false`, never an error, for an anonymous request, an unknown key or a user of another client. You own and apply the policy; the kit only supplies the helpers.

The helpers live in the exposed `auth_kit` schema. The tables and implementations live in `auth_kit_private`, which is never added to your project's exposed schemas. Do not add it: nothing in the kit needs it over HTTP, and the kit's acceptance suite asserts that no request can reach it.

```
create policy notes_read on app.notes for select to authenticated using (
  auth_kit.has_permission('example', 'notes:read:any')
  or (auth_kit.has_permission('example', 'notes:read:own') and owner_id = auth.uid())
);
```

Understand what each path checks before choosing it:

	Node path (<code>resolveSession</code>)	Direct path (browser → PostgREST under RLS)
Verifies the token signature and expiry	yes	yes
Asks Supabase Auth whether the session still exists	yes, on every request	no, never
Sees a membership revoke or model change	next request	next request
Sees sign-out, ban or user deletion	next request	only when the access token expires (up to the configured lifetime)
Suitable for	anything sensitive	public or low-risk reads and writes

A user who signed out, or whom you banned or deleted, can keep using the direct path with an unexpired token. The kit calls this "revocation bounded by JWT expiry" and never claims more for that path. Route anything you would not want such a user to see through your Node server. Test revocation on both paths before you go live.

9. Browser usage

Default screens. Mount `createAuthScreens(config)` at your configured route prefix (default `/account`).

Headless. `createAuthController(config)` exposes `state`, `signUp`, `signIn`, `signInWithGoogle`, `signOut`, `requestReset`, `completeReset`, `enrolMfa`, `challengeMfa`, `getPrincipal()` and a typed event stream.

Screen states you should render: `idle`, `submitting`, `sent`, `error(code)`, `expired_link`, `already_used`, `mfa_enrol`, `mfa_challenge`, `setup_pending`, `no_access`, `signed_in`, `offline`.

After sign-in the kit calls `ensure_profile`, reads the user's access, and calls `join_client` only if the user has never been enrolled for this client. Two "no membership" states look different on purpose:

- `setup_pending`: the client is not registered yet, or has no self-assignable role yet. The operator's setup is in progress. Show a retry button; do not loop automatically.
- `no_access`: the client is closed (invite-only), or the user was enrolled once and has since had their membership revoked. Tell the user to ask the site for access. There is nothing to retry.

`getPrincipal()` in the browser is display state only. Never make an authorization decision from it.

10. Errors

Code	Meaning	Suggested HTTP
<code>no_token</code>	no bearer token	401
<code>invalid_token</code>	bad signature, or the session was revoked, banned or deleted	401
<code>expired</code>	token past its lifetime	401
<code>email_unverified</code>	the user has not confirmed their email	403
<code>mfa_required</code>	the permission is held only by a role that needs an MFA-verified session	403, route the user to MFA
<code>forbidden</code>	no active role grants the permission	403
<code>request_conflict</code>	a <code>requestId</code> was reused with a different payload, or by a different actor	409
<code>unavailable</code>	Supabase could not be reached or answered abnormally	503
<code>provider_unavailable</code>	the Google or SMTP flow failed	show retry
<code>config_invalid</code> , <code>model_invalid</code>	your configuration or model file failed validation	fail at startup or in the CLI

CLI-only outcomes:

Code	Command	Meaning
<code>unknown_user</code> , <code>ambiguous_user</code>	<code>bootstrap-manager -email</code>	the email matches no confirmed user, or several
<code>lookup_incomplete</code>	<code>bootstrap-manager -email</code>	the whole user list could not be read; use <code>--user-id</code>
<code>setup_pending</code>	<code>bootstrap-manager -invite</code>	an invitation was sent; rerun with the printed id after the user accepts
<code>request_in_progress</code>	<code>mfa -reset</code>	a run with the same request id is still inside its lease; rerun after it finishes
<code>run_superseded</code>	<code>mfa -reset</code>	this run was taken over after its lease expired and wrote nothing; rerun with the same id to see the outcome
<code>lease_expired</code>	<code>mfa -reset</code>	this run stopped itself before an admin call because its claim could have expired; rerun with the same id

11. Operational notes

- **Scaling out.** The server library is stateless; run as many instances as you like. Sessions live in Supabase; the model lives in Postgres. Each authenticated request costs one Auth call and one database call per instance. All writes for one client (joins, grants, revokes, model changes) are serialised by one

lock; reads are not. This suits small and medium sites; it is not designed for thousands of membership writes per second.

- **Revocation.** On the Node path, sign-out, ban, deletion, membership revoke and model changes take effect on the next request. On the direct path, only membership revoke and model changes are immediate; see section 8.
- **MFA.** A user holding an `mfa_required` role is prompted to enrol. Until they verify, permissions from that role are withheld and their other roles keep working. A user at MFA level can remove their own authenticator from the kit's screens.
- **Resetting a lost authenticator.** The operator runs `auth-kit mfa-reset --user-id <uuid> --request-id <uuid>`. The CLI first binds the request id to that user in the database, then lists and deletes the user's verified authenticators through the admin API, then completes the audit row with the list of authenticators that existed at the start and the list it deleted. It is safe to rerun with the same request id:
 - a completed run returns its stored result and does not touch the user's authenticators again;
 - a run interrupted midway can be rerun with the same id after about two minutes and resumes with the original list;
 - the same id with a different user is refused before anything is deleted;
 - a second run started while the first is still going is refused with `request_in_progress`;
 - a rerun that takes over an interrupted run holds it alone: any further rerun is refused with `request_in_progress` until another two minutes pass, and if the interrupted run was only slow and wakes up, it can no longer record or complete anything (`run_superseded`);
 - a run stops itself with `lease_expired` if it has been going for more than 100 seconds before its next admin call; rerun it with the same id and it resumes.

The reset applies to the user across every client of the project.

- **Free plan.** On Supabase's Free plan, projects pause after a period without activity, and email sending and token refresh are rate limited. Custom SMTP is required for real sign-ups on any plan.
- **Diagnostics.** `auth-kit doctor` reports grant drift, memberships without audit events, model-hash mismatch against your file, and permission keys your code references but the model does not declare (when pointed at your source). Its default mode reads the catalog; `--probe` exercises a disposable user for real.
- **Model recovery.** If the model was changed from the SQL editor, `auth-kit export-model` writes the applied model in file format. Commit it before the next `apply-model`, or that apply will revert the change.

12. Limitations

- Supabase Auth only; no other identity back end.
- Two supported host types: a static site with its own build step, and a Node 22+ server. Other frameworks are possible but untested.
- Type-level RBAC only. No relationship- or attribute-based authorization; ownership is yours.
- Direct database access under RLS does not get the live session check (section 8). The RLS helpers are optional; an application that keeps its data outside Supabase Postgres never uses them.
- No member listing. A consumer that needs a staff directory fills it from resolved sessions.
- No admin UI for the model. No cookie-session mode.
- The project's secret key is root. Keep it with the operator only.
- Supabase project configuration is manual per environment and cannot be automated by install.
- Write throughput per client is serialised (section 11).

13. Checklist before you call the integration done

- ☐ The secret key, Management API token and provider secrets appear nowhere in the browser bundle, logs or repository.
- ☐ Every route that touches sensitive data goes through your Node server: `resolveSession`, then `requirePermission`, then your own ownership rule.
- ☐ Any table exposed on the direct path holds only data a signed-out user may keep seeing until their token expires.
- ☐ Own-rows and all-rows authority use distinct permission keys, checked in the order shown in section 7.
- ☐ `unavailable` returns 503, never a fallback principal.
- ☐ The model file is in the repository and applied, and `doctor` reports no drift.
- ☐ At least one manager was bootstrapped by the operator, and the client is live.
- ☐ Revoking a membership was tested: the user's next sign-in shows `no_access`, not a restored role.
- ☐ Redirect URLs in the project are exact, per environment.
- ☐ Auth pages are `noindex`; public pages work signed out.
- ☐ Historical records are reachable only through explicit link rows your application stores.